

Ejercicios de SQL GymUPV

Campo	Descripcion
Materia	Bases de Datos
Capitulo	Capitulo 3 - Introduction to SQL
DBMS	MySQL
Herramienta sugerida	DBeaver
Base de datos	GymUPV
Total de ejercicios	50

Proposito del material

Este documento integra 50 ejercicios practicos de SQL utilizando la base de datos GymUPV. Los ejercicios estan pensados para trabajarse en DBeaver conectado a un DBMS MySQL local. Se abordan instrucciones del capitulo 3 de SQL: DDL, DML, SELECT, WHERE, alias, ordenamiento, patrones, agregaciones, GROUP BY, HAVING, JOIN, subconsultas, vistas, CASE, CTE, modificaciones y transacciones.

Requisitos para trabajar

- Tener MySQL Server instalado o acceso a un servidor MySQL.
- Tener DBeaver instalado.
- Crear una conexion a MySQL desde DBeaver.
- Ejecutar primero el script de estructura y despues el script de datos de prueba.
- Realizar los ejercicios en un script SQL propio y guardar evidencias.

Estructura de la base de datos GymUPV

Ejecuta el siguiente script en DBeaver. Si ya existe la base de datos GymUPV, el script la elimina y la vuelve a crear. Usalo solo en ambiente de practica.

```
DROP DATABASE IF EXISTS GymUPV;
CREATE DATABASE GymUPV CHARACTER SET utf8mb4 COLLATE utf8mb4_spanish_ci;
USE GymUPV;
```

```
CREATE TABLE Membresias (
  id_membresia INT PRIMARY KEY AUTO_INCREMENT,
  nombre_membresia VARCHAR(50) NOT NULL,
  descripcion VARCHAR(200),
  costo DECIMAL(10,2) NOT NULL,
  duracion_dias INT NOT NULL,
  activo BOOLEAN NOT NULL DEFAULT TRUE
);
```

```
CREATE TABLE Alumnos (
  id_alumno INT PRIMARY KEY AUTO_INCREMENT,
  matricula VARCHAR(20) NOT NULL UNIQUE,
  nombre VARCHAR(80) NOT NULL,
  apellido_paterno VARCHAR(80) NOT NULL,
  apellido_materno VARCHAR(80),
  fecha_nacimiento DATE,
  sexo CHAR(1),
  carrera VARCHAR(100),
  telefono VARCHAR(20),
  correo VARCHAR(100),
  fecha_registro DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```

    activo BOOLEAN DEFAULT TRUE,
    id_membresia INT,
    FOREIGN KEY (id_membresia) REFERENCES Membresias(id_membresia)
);

```

```

CREATE TABLE Entrenadores (
    id_entrenador INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(80) NOT NULL,
    apellido_paterno VARCHAR(80) NOT NULL,
    apellido_materno VARCHAR(80),
    especialidad VARCHAR(100),
    telefono VARCHAR(20),
    correo VARCHAR(100),
    activo BOOLEAN DEFAULT TRUE
);

```

```

CREATE TABLE Clases (
    id_clase INT PRIMARY KEY AUTO_INCREMENT,
    nombre_clase VARCHAR(100) NOT NULL,
    descripcion VARCHAR(200),
    horario VARCHAR(100),
    capacidad INT,
    id_entrenador INT,
    FOREIGN KEY (id_entrenador) REFERENCES
Entrenadores(id_entrenador) );

```

```

CREATE TABLE ClaseAlumnos (
    id_clase_alumno INT PRIMARY KEY AUTO_INCREMENT,
    id_clase INT NOT NULL,
    id_alumno INT NOT NULL,
    fecha_inscripcion DATE NOT NULL,
    estatus VARCHAR(30) DEFAULT 'Activa',
    FOREIGN KEY (id_clase) REFERENCES Clases(id_clase),
    FOREIGN KEY (id_alumno) REFERENCES
Alumnos(id_alumno) );

```

```

CREATE TABLE Rutinas (
    id_rutina INT PRIMARY KEY AUTO_INCREMENT,
    id_alumno INT NOT NULL,
    id_entrenador INT NOT NULL,
    nombre_rutina VARCHAR(100) NOT NULL,
    objetivo VARCHAR(200),
    fecha_asignacion DATE,
    FOREIGN KEY (id_alumno) REFERENCES Alumnos(id_alumno), FOREIGN
KEY (id_entrenador) REFERENCES Entrenadores(id_entrenador) );

```

```

CREATE TABLE Ejercicios (
    id_ejercicio INT PRIMARY KEY AUTO_INCREMENT,
    nombre_ejercicio VARCHAR(100) NOT NULL,
    descripcion VARCHAR(300),
    grupo_muscular VARCHAR(100)
);

```

```

CREATE TABLE RutinaEjercicios (
    id_rutina_ejercicio INT PRIMARY KEY
AUTO_INCREMENT, id_rutina INT NOT NULL,
    id_ejercicio INT NOT NULL,
    series INT,
    repeticiones INT,
    descanso_segundos INT,
    FOREIGN KEY (id_rutina) REFERENCES Rutinas(id_rutina),
    FOREIGN KEY (id_ejercicio) REFERENCES
Ejercicios(id_ejercicio) );

```

```

CREATE TABLE EquiposGym (
    id_equipo INT PRIMARY KEY AUTO_INCREMENT,
    nombre_equipo VARCHAR(100) NOT NULL,
    marca VARCHAR(100),
    fecha_compra DATE,
    estado VARCHAR(50),
    ubicacion VARCHAR(100),
    mantenimiento_requerido BOOLEAN DEFAULT FALSE
);

```

```

CREATE TABLE EjercicioEquipos (
    id_ejercicio_equipo INT PRIMARY KEY

```

```

AUTO_INCREMENT, id_ejercicio INT NOT NULL,
id_equipo INT NOT NULL,
FOREIGN KEY (id_ejercicio) REFERENCES
Ejercicios(id_ejercicio), FOREIGN KEY (id_equipo) REFERENCES
EquiposGym(id_equipo) );

```

```

CREATE TABLE Asistencias (
id_asistencia INT PRIMARY KEY AUTO_INCREMENT,
id_alumno INT NOT NULL,
fecha_entrada DATETIME NOT NULL,
fecha_salida DATETIME,
observaciones VARCHAR(200),
FOREIGN KEY (id_alumno) REFERENCES
Alumnos(id_alumno) );

```

```

CREATE TABLE Pagos (
id_pago INT PRIMARY KEY AUTO_INCREMENT,
id_alumno INT NOT NULL,
fecha_pago DATE NOT NULL,
monto DECIMAL(10,2) NOT NULL,
metodo_pago VARCHAR(50),
referencia VARCHAR(100),
FOREIGN KEY (id_alumno) REFERENCES
Alumnos(id_alumno) );

```

Datos de prueba

Despues de crear las tablas, ejecuta los siguientes INSERT para contar con datos suficientes para las practicas.

```
USE GymUPV;
```

```

INSERT INTO Membresias (nombre_membresia, descripcion, costo, duracion_dias,
activo) VALUES ('Mensual', 'Acceso general por 30 dias', 500.00, 30, TRUE),
('Premium', 'Acceso general, clases y rutinas personalizadas', 800.00, 30, TRUE),
('Semestral', 'Acceso por seis meses', 2500.00, 180, TRUE),
('Visitante', 'Acceso de un dia', 80.00, 1, TRUE);

```

```

INSERT INTO Alumnos (matricula, nombre, apellido_paterno, apellido_materno,
fecha_nacimiento, sexo, carrera, telefono, correo, id_membresia) VALUES
('2025001','Ana','Lopez','Garcia','2005-01-10','F','ITI','8341000001','ana.lopez@upv
.edu.mx',2), ('2025002','Juan','Perez','Martinez','2004-07-
15','M','Mecatronica','8341000002','juan.perez@upv.edu.mx',1),
('2025003','Maria','Ruiz','Torres','2006-03-
20','F','Industrial','8341000003','maria.ruiz@upv.edu.mx',2),
('2025004','Luis','Hernandez','Diaz','2003-09-05','M','ITI','8341000004','luis.hernandez@upv
.edu.mx',3), ('2025005','Sofia','Gomez','Luna','2005-12-
02','F','Administracion','8341000005','sofia.gomez@upv.edu.mx',1),
('2025006','Diego','Ramirez','Soto','2004-11-
11','M','Energia','8341000006','diego.ramirez@upv.edu.mx',4);

```

```

INSERT INTO Entrenadores (nombre, apellido_paterno, apellido_materno, especialidad,
telefono, correo) VALUES
('Carlos','Diaz','Mendez','Funcional','8342000001','carlos.diaz@upv.edu.mx'),
('Laura','Gomez','Perez','Yoga','8342000002','laura.gomez@upv.edu.mx'),
('Roberto','Sanchez','Cruz','Fuerza','8342000003','roberto.sanchez@upv.edu.mx'),
('Elena','Morales','Rios','Cardio','8342000004','elena.morales@upv.edu.mx');

```

```

INSERT INTO Clases (nombre_clase, descripcion, horario, capacidad,
id_entrenador) VALUES ('Spinning','Clase de bicicleta fija','Lunes y
miercoles 07:00',20,4),
('Yoga','Flexibilidad y respiracion','Martes y jueves 08:00',15,2),
('Funcional','Circuitos de fuerza y resistencia','Lunes y viernes 18:00',25,1),
('Fuerza','Entrenamiento con pesas','Miercoles 17:00',12,3);

```

```

INSERT INTO ClaseAlumnos (id_clase, id_alumno, fecha_inscripcion, estatus) VALUES
(1,1,'2026-05-01','Activa'),(1,2,'2026-05-01','Activa'),(2,3,'2026-05-02','Activa'),
(3,4,'2026-05-03','Activa'),(3,5,'2026-05-03','Baja'),(4,1,'2026-05-04','Activa'),(4,
6,'2026-05-04','Activa');

```

```

INSERT INTO Rutinas (id_alumno, id_entrenador, nombre_rutina, objetivo,
fecha_asignacion) VALUES (1,3,'Pierna intensa','Hipertrofia de tren
inferior','2026-05-05'),
(2,1,'Cardio inicial','Mejora de condicion fisica','2026-05-05'),
(3,2,'Movilidad y core','Flexibilidad y abdomen','2026-05-06'),
(4,3,'Fuerza basica','Incrementar fuerza general','2026-05-06'),

```

```

(6,4,'Cardio avanzado','Resistencia cardiovascular','2026-05-07');

INSERT INTO Ejercicios (nombre_ejercicio, descripcion, grupo_muscular) VALUES
('Sentadilla','Flexion de rodillas con peso corporal o barra','Pierna'),
('Prensa','Empuje en maquina de prensa','Pierna'),
('Plancha','Sostener posicion abdominal','Core'),
('Burpees','Ejercicio de cuerpo completo','Cardio'),
('Press banca','Empuje horizontal con barra','Pecho'),
('Remo','Traccion para espalda','Espalda'),
('Curl biceps','Flexion de codo con mancuerna','Biceps'),
('Peso muerto','Levantamiento de cadena posterior','Espalda/Pierna');

INSERT INTO RutinaEjercicios (id_rutina, id_ejercicio, series, repeticiones,
descanso_segundos) VALUES (1,1,4,12,90), (1,2,4,15,90), (1,8,3,8,120),
(2,4,5,10,60), (2,3,3,30,45),
(3,3,4,40,45), (3,4,3,12,60),
(4,5,5,8,120), (4,6,4,10,90), (4,8,4,6,150),
(5,4,6,15,45), (5,3,4,45,30);
INSERT INTO EquiposGym (nombre_equipo, marca, fecha_compra, estado, ubicacion,
mantenimiento_requerido) VALUES
('Bicicleta fija','FitPro','2025-01-15','Bueno','Sala cardio',FALSE),
('Banco olimpico','IronMax','2024-08-20','Bueno','Area pesas',FALSE),
('Prensa 45','ForceLine','2023-04-10','Regular','Area pesas',TRUE),
('Rack sentadilla','IronMax','2024-02-12','Bueno','Area pesas',FALSE),
('Caminadora','RunFast','2025-03-01','Bueno','Sala cardio',FALSE);

INSERT INTO EjercicioEquipos (id_ejercicio, id_equipo) VALUES
(1,4), (2,3), (5,2), (4,5), (8,4), (6,2);

INSERT INTO Asistencias (id_alumno, fecha_entrada, fecha_salida,
observaciones) VALUES (1,'2026-05-10 07:00:00','2026-05-10
08:10:00','Entrenamiento completo'),
(2,'2026-05-10 07:05:00','2026-05-10 07:50:00','Cardio'),
(3,'2026-05-11 08:00:00','2026-05-11 09:00:00','Yoga'),
(4,'2026-05-12 18:00:00','2026-05-12 19:20:00','Fuerza'),
(1,'2026-05-13 17:00:00','2026-05-13 18:30:00','Fuerza'),
(6,'2026-05-14 07:30:00',NULL,'Salida no registrada');

INSERT INTO Pagos (id_alumno, fecha_pago, monto, metodo_pago,
referencia) VALUES (1,'2026-05-01',800.00,'Tarjeta','PAG-001'),
(2,'2026-05-01',500.00,'Efectivo','PAG-002'),
(3,'2026-05-02',800.00,'Transferencia','PAG-003'),
(4,'2026-05-03',2500.00,'Tarjeta','PAG-004'),
(5,'2026-05-04',500.00,'Efectivo','PAG-005'),
(6,'2026-05-05',80.00,'Efectivo','PAG-006');

```

Resumen de temas incluidos

Rango	Tema principal
1-15	SELECT, alias, DISTINCT, WHERE, LIKE, BETWEEN, ORDER BY, LIMIT, NULL
16-25	Funciones de agregacion, GROUP BY y HAVING
26-35	JOIN en relaciones 1:N y N:M con tablas puente
36-40	Subconsultas, IN, NOT IN, EXISTS y comparaciones
41-45	INSERT, UPDATE y DELETE
46-50	VIEW, CASE, CTE y transacciones

Instrucciones:

- Resuelve cada ejercicio escribiendo el SQL correspondiente.
- Ejecuta la consulta en DBeaver y valida los resultados.
- No modifiques el script base salvo en ejercicios de INSERT, UPDATE y DELETE.
- En ejercicios de modificacion, puedes volver a ejecutar el script base si necesitas restaurar los datos.
- Entrega el

archivo .sql con tus respuestas y capturas de resultados solicitadas.

50 ejercicios practicos

Ejercicio 1 - SELECT basico

Muestra todos los registros de la tabla Alumnos.

Escribe aqui tu consulta SQL:

```
SELECT * FROM Alumnos;
```

1	2,2025001,Ana,Lopez,Garcia,2005-01-10,F,ITI,8341000001,ana.lopez@upv.edu.mx,2026-06-08 22:42:18,1,2
2	3,2025002,Juan,Perez,Martinez,2004-07-15,M,Mecatronica,8341000002,juan.perez@upv.edu.mx,2026-06-08 22:42:18,1,1
3	4,2025003,Maria,Ruiz,Torres,2006-03-20,F,Industrial,8341000003,maria.ruiz@upv.edu.mx,2026-06-08 22:42:18,1,2
4	5,2025004,Luis,Hernandez,Diaz,2003-09-05,M,ITI,8341000004,luis.hernandez@upv.edu.mx,2026-06-08 22:42:18,1,3
5	6,2025005,Sofia,Gomez,Luna,2005-12-02,F,Administracion,8341000005,sofia.gomez@upv.edu.mx,2026-06-08 22:42:18,1,1
6	7,2025006,Diego,Ramirez,Soto,2004-11-11,M,Energia,8341000006,diego.ramirez@upv.edu.mx,2026-06-08 22:42:18,1,4

Ejercicio 2 - SELECT columnas

Muestra matricula, nombre, apellido_paterno y carrera de todos los

alumnos. Escribe aqui tu consulta SQL:

```
SELECT matricula, nombre, apellido_paterno, carrera  
FROM Alumnos;
```

1	2025001,Ana,Lopez,ITI
2	2025002,Juan,Perez,Mecatronica
3	2025003,Maria,Ruiz,Industrial
4	2025004,Luis,Hernandez,ITI
5	2025005,Sofia,Gomez,Administracion
6	2025006,Diego,Ramirez,Energia

Ejercicio 3 - Alias con AS

Muestra nombre y apellido_paterno usando alias Alumno y Apellido.

Escribe aqui tu consulta SQL:

```
SELECT nombre AS Alumno,  
        apellido_paterno AS Apellido  
FROM Alumnos;
```

1	Ana,Lopez
2	Juan,Perez
3	Maria,Ruiz
4	Luis,Hernandez
5	Sofia,Gomez
6	Diego,Ramirez

Ejercicio 4 - DISTINCT

Muestra las carreras diferentes registradas en Alumnos.

Escribe aqui tu consulta SQL:

```
SELECT DISTINCT carrera
FROM Alumnos;
```

1	ITI
2	Mecatronica
3	Industrial
4	Administracion
5	Energia

Ejercicio 5 - WHERE igualdad

Muestra los alumnos de la carrera ITI.

Escribe aqui tu consulta SQL:

```
SELECT *
FROM Alumnos
WHERE carrera = 'ITI';
```

1	2,2025001,Ana,Lopez,Garcia,2005-01-10,F,ITI,8341000001,ana.lopez@upv.edu.mx,2026-06-08 22:42:18,1,2
2	5,2025004,Luis,Hernandez,Diaz,2003-09-05,M,ITI,8341000004,luis.hernandez@upv.edu.mx,2026-06-08 22:42:18,1,3

Observacion: prueba la consulta y revisa que el resultado coincida con lo solicitado.

Ejercicio 6 - WHERE con operadores

Muestra las membresias con costo mayor o igual a 800.

Escribe aqui tu consulta SQL:

```
SELECT *
FROM Membresias
```

```
WHERE costo >= 800;
```

1	2,Premium,"Acceso general, clases y rutinas personalizadas",800.00,30,1
2	3,Semestral,Acceso por seis meses,2500.00,180,1
3	6,Premium,"Acceso general, clases y rutinas personalizadas",800.00,30,1
4	7,Semestral,Acceso por seis meses,2500.00,180,1
5	10,Premium,"Acceso general, clases y rutinas personalizadas",800.00,30,1
6	11,Semestral,Acceso por seis meses,2500.00,180,1

Ejercicio 7 - AND

Muestra alumnas activas de la carrera ITI.

Escribe aqui tu consulta SQL:

```
SELECT *  
  
FROM Alumnos  
  
WHERE sexo = 'F'  
  
AND carrera = 'ITI'  
  
AND activo = TRUE;
```

1	2,2025001,Ana,Lopez,Garcia,2005-01-10,F,ITI,8341000001,ana.lopez@upv.edu.mx,2026-06-08 22:42:18,1,2
2	

Ejercicio 8 - OR

Muestra alumnos de ITI o Mecatronica.

Escribe aqui tu consulta SQL:

```
SELECT *  
  
FROM Alumnos  
  
WHERE carrera = 'ITI'  
  
OR carrera = 'Mecatronica';
```

1	2,2025001,Ana,Lopez,Garcia,2005-01-10,F,ITI,8341000001,ana.lopez@upv.edu.mx,2026-06-08 22:42:18,1,2
2	3,2025002,Juan,Perez,Martinez,2004-07-15,M,Mecatronica,8341000002,juan.perez@upv.edu.mx,2026-06-08 22:42:18,1,1
3	5,2025004,Luis,Hernandez,Diaz,2003-09-05,M,ITI,8341000004,luis.hernandez@upv.edu.mx,2026-06-08 22:42:18,1,3

Ejercicio 9 - LIKE

Muestra alumnos cuyo correo contenga upv.edu.mx.

Escribe aqui tu consulta SQL:

```
SELECT *  
  
FROM Alumnos  
  
WHERE correo LIKE '%upv.edu.mx%';
```

1	2,2025001,Ana,Lopez,Garcia,2005-01-10,F,ITI,8341000001,ana.lopez@upv.edu.mx,2026-06-08 22:42:18,1,2
2	3,2025002,Juan,Perez,Martinez,2004-07-15,M,Mecatronica,8341000002,juan.perez@upv.edu.mx,2026-06-08 22:42:18,1,1
3	4,2025003,Maria,Ruiz,Torres,2006-03-20,F,Industrial,8341000003,maria.ruiz@upv.edu.mx,2026-06-08 22:42:18,1,2
4	5,2025004,Luis,Hernandez,Diaz,2003-09-05,M,ITI,8341000004,luis.hernandez@upv.edu.mx,2026-06-08 22:42:18,1,3
5	6,2025005,Sofia,Gomez,Luna,2005-12-02,F,Administracion,8341000005,sofia.gomez@upv.edu.mx,2026-06-08 22:42:18,1,1
6	7,2025006,Diego,Ramirez,Soto,2004-11-11,M,Energia,8341000006,diego.ramirez@upv.edu.mx,2026-06-08 22:42:18,1,4

Ejercicio 10 - LIKE con patron

Muestra alumnos cuyo apellido paterno inicie con G.

Escribe aqui tu consulta SQL:

```
SELECT *
FROM Alumnos
WHERE apellido_paterno LIKE 'G%';
```

1	6,2025005,Sofia,Gomez,Luna,2005-12-02,F,Administracion,8341000005,sofia.gomez@upv.edu.mx,2026-06-08 22:42:18,1,1
---	--

Observacion: prueba la consulta y revisa que el resultado coincida con lo solicitado.

Ejercicio 11 - BETWEEN

Muestra pagos con monto entre 500 y 1000.

Escribe aqui tu consulta SQL:

```
SELECT *
FROM Pagos
WHERE monto BETWEEN 500 AND 1000;
```

Ejercicio 12 - ORDER BY ascendente

Muestra alumnos ordenados por apellido_paterno y nombre.

Escribe aqui tu consulta SQL:

```
SELECT *
FROM Alumnos
ORDER BY apellido_paterno, nombre;
```

1	6,2025005,Sofia,Gomez,Luna,2005-12-02,F,Administracion,8341000005,sofia.gomez@upv.edu.mx,2026-06-08 22:42:18,1,1
2	5,2025004,Luis,Hernandez,Diaz,2003-09-05,M,ITI,8341000004,luis.hernandez@upv.edu.mx,2026-06-08 22:42:18,1,3
3	2,2025001,Ana,Lopez,Garcia,2005-01-10,F,ITI,8341000001,ana.lopez@upv.edu.mx,2026-06-08 22:42:18,1,2
4	3,2025002,Juan,Perez,Martinez,2004-07-15,M,Mecatronica,8341000002,juan.perez@upv.edu.mx,2026-06-08 22:42:18,1,1
5	7,2025006,Diego,Ramirez,Soto,2004-11-11,M,Energia,8341000006,diego.ramirez@upv.edu.mx,2026-06-08 22:42:18,1,4
6	4,2025003,Maria,Ruiz,Torres,2006-03-20,F,Industrial,8341000003,maria.ruiz@upv.edu.mx,2026-06-08 22:42:18,1,2

Ejercicio 13 - ORDER BY descendente

Muestra pagos ordenados del monto mayor al menor.

Escribe aqui tu consulta SQL:


```
SELECT *
FROM Pagos
ORDER BY monto DESC;
```

	id_pago	id_alumno	fecha_pago	monto	metodo_pago	referencia
1	4	4	2026-05-03	2500.00	Tarjeta	PAG-004
2	1	1	2026-05-01	800.00	Tarjeta	PAG-001
3	3	3	2026-05-02	800.00	Transferencia	PAG-003
4	2	2	2026-05-01	500.00	Efectivo	PAG-002
5	5	5	2026-05-04	500.00	Efectivo	PAG-005
6	6	6	2026-05-05	80.00	Efectivo	PAG-006

Ejercicio 14 - LIMIT

Muestra los tres pagos de mayor monto.

Escribe aqui tu consulta SQL:

```
SELECT *
FROM Pagos
ORDER BY monto DESC
LIMIT 3;
```

	id_pago	id_alumno	fecha_pago	monto	metodo_pago	referencia
1	4	4	2026-05-03	2500.00	Tarjeta	PAG-004
2	1	1	2026-05-01	800.00	Tarjeta	PAG-001
3	3	3	2026-05-02	800.00	Transferencia	PAG-003

Ejercicio 15 - IS NULL

Muestra asistencias donde no se registro fecha_salida.

Escribe aqui tu consulta SQL:

```
SELECT *
FROM Asistencias
WHERE fecha_salida IS NULL;
```

	id_asistencia	id_alumno	fecha_entrada	fecha_salida	observaciones
1	6	6	2026-05-14 07:30:00	<null>	Salida no registrada

Observacion: prueba la consulta y revisa que el resultado coincida con lo solicitado.

Ejercicio 16 - COUNT

Cuenta cuantos alumnos hay registrados.

Escribe aqui tu consulta SQL:

```
SELECT COUNT(*) AS total_alumnos
```

```
FROM Alumnos;
```

total_alumnos	
1	6

Ejercicio 17 - COUNT con WHERE

Cuenta cuantos alumnos activos hay.

Escribe aqui tu consulta SQL:

```
SELECT COUNT(*) AS alumnos_activos  
FROM Alumnos  
WHERE activo = TRUE;
```

alumnos_activos	
1	6

Ejercicio 18 - SUM

Calcula el total recaudado en pagos.

Escribe aqui tu consulta SQL:

```
SELECT SUM(monto) AS total_recaudado  
FROM Pagos;
```

total_recaudado	
1	5180.00

Ejercicio 19 - AVG

Calcula el monto promedio de los pagos.

Escribe aqui tu consulta SQL:

```
SELECT AVG(monto) AS promedio_pagos  
FROM Pagos;
```

promedio_pagos	
1	863.333333

Ejercicio 20 - MIN y MAX

Muestra el pago menor y el pago mayor.

Escribe aqui tu consulta SQL:

```
SELECT MIN(monto) AS pago_menor,
        MAX(monto) AS pago_mayor
FROM Pagos;
```

	pago_menor	pago_mayor
1	80.00	2500.00

Observacion: prueba la consulta y revisa que el resultado coincida con lo solicitado.

Ejercicio 21 - GROUP BY

Muestra total pagado por metodo_pago.

Escribe aqui tu consulta SQL:

```
SELECT metodo_pago,
        SUM(monto) AS total_pagado
FROM Pagos
GROUP BY metodo_pago;
```

	metodo_pago	total_pagado
1	Efectivo	1080.00
2	Tarjeta	3300.00
3	Transferencia	800.00

Ejercicio 22 - GROUP BY COUNT

Cuenta alumnos por carrera.

Escribe aqui tu consulta SQL:

```
SELECT carrera,
        COUNT(*) AS total_alumnos
FROM Alumnos
GROUP BY carrera;
```

	carrera	total_alumnos
1	Administracion	1
2	Energia	1
3	Industrial	1
4	ITI	2
5	Mecatronica	1

Ejercicio 23 - GROUP BY con JOIN

Cuenta alumnos por tipo de membresia.

Escribe aqui tu consulta SQL:

```
SELECT m.nombre_membresia,
       COUNT(a.id_alumno) AS total_alumnos
FROM Membresias m
INNER JOIN Alumnos a
ON m.id_membresia = a.id_membresia
GROUP BY m.nombre_membresia;
```

	nombre_membresia	total_alumnos
1	Mensual	2
2	Premium	2
3	Semestral	1
4	Visitante	1

Ejercicio 24 - HAVING

Muestra metodos de pago cuyo total recaudado sea mayor a 700.

Escribe aqui tu consulta SQL:

```
SELECT metodo_pago,
       SUM(monto) AS total_recaudado
FROM Pagos
GROUP BY metodo_pago
HAVING SUM(monto) > 700;
```

	metodo_pago	total_recaudado
1	Efectivo	1080.00
2	Tarjeta	3300.00
3	Transferencia	800.00

Ejercicio 25 - GROUP BY y ORDER BY

Muestra total recaudado por metodo_pago ordenado de mayor a menor.

Escribe aqui tu consulta SQL:

```
SELECT metodo_pago,
       SUM(monto) AS total_recaudado
FROM Pagos
GROUP BY metodo_pago
ORDER BY total_recaudado DESC;
```

	metodo_pago	total_recaudado
1	Tarjeta	3300.00
2	Efectivo	1080.00
3	Transferencia	800.00

Observacion: prueba la consulta y revisa que el resultado coincida con lo solicitado.

Ejercicio 26 - JOIN 1:N

Muestra alumnos con su nombre de membresia y costo.

Escribe aqui tu consulta SQL:

```
SELECT a.nombre,
       a.apellido_paterno,
       m.nombre_membresia,
       m.costo
FROM Alumnos a
INNER JOIN Membresias m
ON a.id_membresia = m.id_membresia;
```

	nombre	apellido_paterno	nombre_membresia	costo
1	Ana	Lopez	Premium	800.00
2	Juan	Perez	Mensual	500.00
3	Maria	Ruiz	Premium	800.00
4	Luis	Hernandez	Semestral	2500.00
5	Sofia	Gomez	Mensual	500.00
6	Diego	Ramirez	Visitante	80.00

Ejercicio 27 - JOIN 1:N

Muestra clases con el entrenador asignado.

Escribe aqui tu consulta SQL:

```
SELECT c.nombre_clase,
       e.nombre,
       e.apellido_paterno
FROM Clases c
INNER JOIN Entrenadores e
ON c.id_entrenador = e.id_entrenador;
```

	nombre_clase	nombre	apellido_paterno
1	Funcional	Carlos	Diaz
2	Yoga	Laura	Gomez
3	Fuerza	Roberto	Sanchez
4	Spinning	Elena	Morales

Ejercicio 28 - JOIN con filtro

Muestra clases impartidas por entrenadores de especialidad Cardio.

Escribe aqui tu consulta SQL:

```
SELECT c.nombre_clase
FROM Clases c
INNER JOIN Entrenadores e
ON c.id_entrenador = e.id_entrenador
WHERE e.especialidad = 'Cardio';
```

	nombre_clase
1	Spinning

Ejercicio 29 - JOIN N:M tabla puente

Muestra alumnos inscritos en cada clase.

Escribe aqui tu consulta SQL:

```
SELECT c.nombre_clase,
       a.nombre,
       a.apellido_paterno
FROM ClaseAlumnos ca
INNER JOIN Clases c
ON ca.id_clase = c.id_clase
INNER JOIN Alumnos a
ON ca.id_alumno = a.id_alumno;
```

	nombre_clase	nombre	apellido_paterno
1	Spinning	Ana	Lopez
2	Spinning	Juan	Perez
3	Yoga	Maria	Ruiz
4	Funcional	Luis	Hernandez
5	Funcional	Sofia	Gomez
6	Fuerza	Ana	Lopez
7	Fuerza	Diego	Ramirez

Ejercicio 30 - JOIN N:M tabla puente

Muestra ejercicios de cada rutina con series y repeticiones.

Escribe aqui tu consulta SQL:

```
SELECT r.nombre_rutina,
       e.nombre_ejercicio,
       re.series,
       re.repeticiones
FROM RutinaEjercicios re
INNER JOIN Rutinas r
ON re.id_rutina = r.id_rutina
INNER JOIN Ejercicios e
```

```
ON re.id_ejercicio = e.id_ejercicio;
```

	nombre_rutina	nombre_ejercicio	series	repeticiones
1	Pierna intensa	Sentadilla	4	12
2	Pierna intensa	Prensa	4	15
3	Pierna intensa	Peso muerto	3	8
4	Cardio inicial	Burpees	5	10
5	Cardio inicial	Plancha	3	30
6	Movilidad y core	Plancha	4	40
7	Movilidad y core	Burpees	3	12
8	Fuerza basica	Press banca	5	8
9	Fuerza basica	Remo	4	10
10	Fuerza basica	Peso muerto	4	6
11	Cardio avanzado	Burpees	6	15
12	Cardio avanzado	Plancha	4	45

Observacion: prueba la consulta y revisa que el resultado coincida con lo solicitado.

Ejercicio 31 - JOIN multiple

Muestra alumno, rutina, entrenador que la asigno y objetivo.

Escribe aqui tu consulta SQL:

```
SELECT a.nombre,
       r.nombre_rutina,
       e.nombre AS entrenador,
       r.objetivo
FROM Rutinas r
INNER JOIN Alumnos a
ON r.id_alumno = a.id_alumno
INNER JOIN Entrenadores e
ON r.id_entrenador = e.id_entrenador;
```

	nombre	nombre_rutina	entrenador	objetivo
1	Juan	Cardio inicial	Carlos	Mejora de condicion fisica
2	Maria	Movilidad y core	Laura	Flexibilidad y abdomen
3	Ana	Pierna intensa	Roberto	Hipertrofia de tren inferior
4	Luis	Fuerza basica	Roberto	Incrementar fuerza general
5	Diego	Cardio avanzado	Elena	Resistencia cardiovascular

Ejercicio 32 - LEFT JOIN

Muestra todas las membresias aunque no tengan alumnos asignados.

Escribe aqui tu consulta SQL:

```
SELECT m.nombre_membresia,  
       a.nombre  
FROM Membresias m  
LEFT JOIN Alumnos a  
ON m.id_membresia = a.id_membresia;
```

	nombre_membresia	nombre
1	Mensual	Juan
2	Mensual	Sofia
3	Premium	Ana
4	Premium	Maria
5	Semestral	Luis
6	Visitante	Diego

Ejercicio 33 - LEFT JOIN con conteo

Muestra cuantas rutinas tiene cada alumno, incluyendo alumnos sin rutinas. Escribe aqui tu consulta SQL:

```
SELECT a.nombre,  
       COUNT(r.id_rutina) AS total_rutinas  
FROM Alumnos a  
LEFT JOIN Rutinas r  
ON a.id_alumno = r.id_alumno  
GROUP BY a.id_alumno, a.nombre;
```

	nombre	total rutinas
1	Ana	1
2	Juan	1
3	Maria	1
4	Luis	1
5	Sofia	0
6	Diego	1

Ejercicio 34 - JOIN equipos

Muestra que equipos se usan por ejercicio.

Escribe aqui tu consulta SQL:

```
SELECT e.nombre_ejercicio,
       eq.nombre_equipo
FROM EjercicioEquipos ee
INNER JOIN Ejercicios e
ON ee.id_ejercicio = e.id_ejercicio
INNER JOIN EquiposGym eq
ON ee.id_equipo = eq.id_equipo;
```

	nombre_ejercicio	nombre_equipo
1	Press banca	Banco olimpico
2	Remo	Banco olimpico
3	Prensa	Prensa 45
4	Sentadilla	Rack sentadilla
5	Peso muerto	Rack sentadilla
6	Burpees	Caminadora

Ejercicio 35 - JOIN asistencias

Muestra nombre del alumno, fecha_entrada y fecha_salida de sus

asistencias. Escribe aqui tu consulta SQL:

```
SELECT a.nombre,
```

```

asi.fecha_entrada,

asi.fecha_salida

FROM Asistencias asi

INNER JOIN Alumnos a

ON asi.id_alumno = a.id_alumno;

```

	nombre	fecha_entrada	fecha_salida
1	Ana	2026-05-10 07:00:00	2026-05-10 08:10:00
2	Juan	2026-05-10 07:05:00	2026-05-10 07:50:00
3	Maria	2026-05-11 08:00:00	2026-05-11 09:00:00
4	Luis	2026-05-12 18:00:00	2026-05-12 19:20:00
5	Ana	2026-05-13 17:00:00	2026-05-13 18:30:00
6	Diego	2026-05-14 07:30:00	<null>

Observacion: prueba la consulta y revisa que el resultado coincida con lo solicitado.

Ejercicio 36 - Subconsulta WHERE IN

Muestra alumnos que tienen al menos una rutina asignada.

Escribe aqui tu consulta SQL:

```

SELECT *

FROM Alumnos

WHERE id_alumno IN (

    SELECT id_alumno

    FROM Rutinas

);

```

```

{"id_alumno": 1, "matricula": "2025001", "nombre": "Ana", "apellido_paterno": "Lopez", "apellido_materno": "Garcia", "fecha_nacimiento": "2005-01-10", "sexo": "F", "carrera": "Medicina"},
{"id_alumno": 2, "matricula": "2025002", "nombre": "Juan", "apellido_paterno": "Perez", "apellido_materno": "Martinez", "fecha_nacimiento": "2004-07-15", "sexo": "M", "carrera": "Ingenieria"},
{"id_alumno": 3, "matricula": "2025003", "nombre": "Maria", "apellido_paterno": "Ruiz", "apellido_materno": "Torres", "fecha_nacimiento": "2006-03-20", "sexo": "F", "carrera": "Derecho"},
{"id_alumno": 4, "matricula": "2025004", "nombre": "Luis", "apellido_paterno": "Hernandez", "apellido_materno": "Diaz", "fecha_nacimiento": "2003-09-05", "sexo": "M", "carrera": "Ciencias"},
{"id_alumno": 6, "matricula": "2025006", "nombre": "Diego", "apellido_paterno": "Ramirez", "apellido_materno": "Soto", "fecha_nacimiento": "2004-11-11", "sexo": "M", "carrera": "Arquitectura"}

```

Ejercicio 37 - Subconsulta NOT IN

Muestra alumnos que no estan inscritos en ninguna clase.

Escribe aqui tu consulta SQL:

```
SELECT *
FROM Alumnos
WHERE id_alumno NOT IN (
    SELECT id_alumno
    FROM ClaseAlumnos
);
```

Ejercicio 38 - Subconsulta con comparacion

Muestra pagos cuyo monto es mayor al promedio de pagos.

Escribe aqui tu consulta SQL:

```
SELECT *
FROM Pagos
WHERE monto > (
    SELECT AVG(monto)
    FROM Pagos
);
```

	id_pago	id_alumno	fecha_pago	monto	metodo_pago	referencia
1		4	2026-05-03	2500.00	Tarjeta	PAG-004

Ejercicio 39 - EXISTS

Muestra entrenadores que tienen al menos una clase asignada.

Escribe aqui tu consulta SQL:

```
SELECT *
FROM Entrenadores e
WHERE EXISTS (
    SELECT 1
    FROM Clases c
    WHERE c.id_entrenador = e.id_entrenador
);
```

	id_entrenador	nombre	apellido_paterno	apellido_materno	especialidad	telefono	correo	activo
1		1 Carlos	Diaz	Mendez	Funcional	8342000001	carlos.diaz@upv.edu.mx	
2		2 Laura	Gomez	Perez	Yoga	8342000002	laura.gomez@upv.edu.mx	
3		3 Roberto	Sanchez	Cruz	Fuerza	8342000003	roberto.sanchez@upv.edu.mx	
4		4 Elena	Morales	Rios	Cardio	8342000004	elena.morales@upv.edu.mx	

Ejercicio 40 - Subconsulta correlacionada

Muestra alumnos que han pagado un monto mayor a 700 en algun pago.

Escribe aqui tu consulta SQL:

```
SELECT *
FROM Alumnos a
WHERE EXISTS (
    SELECT 1
    FROM Pagos p
    WHERE p.id_alumno = a.id_alumno
    AND p.monto > 700
);
```

	id_alumno	matricula	nombre	apellido_paterno	apellido_materno	fecha_nacimiento	sexo	carrera	
1	1	2025001	Ana	Lopez	Garcia	2005-01-10	F	ITI	83
2	3	2025003	Maria	Ruiz	Torres	2006-03-20	F	Industrial	83
3	4	2025004	Luis	Hernandez	Diaz	2003-09-05	M	ITI	83

Observacion: prueba la consulta y revisa que el resultado coincida con lo solicitado.

Ejercicio 41 - INSERT

Inserta una nueva membresia llamada Anual con costo 4500 y duracion 365

dias. Escribe aqui tu consulta SQL:

Ejercicio 42 - INSERT con FK

Inserta un nuevo alumno asociado a la membresia Mensual.

Escribe aqui tu consulta SQL:

Ejercicio 43 - UPDATE

Actualiza el telefono del alumno con matricula 2025002.

Escribe aqui tu consulta SQL:

Ejercicio 44 - UPDATE con condicion

Aumenta 10% el costo de las membresias activas con costo menor a 1000.

Escribe aqui tu consulta SQL:

Ejercicio 45 - DELETE

Elimina una inscripcion dada de baja en ClaseAlumnos.

Escribe aqui tu consulta SQL:

Observacion: prueba la consulta y revisa que el resultado coincida con lo solicitado.

Ejercicio 46 - CREATE VIEW

Crea una vista que muestre alumno, carrera, membresia y costo.

Escribe aqui tu consulta SQL:

Ejercicio 47 - Consulta de vista

Consulta todos los registros de la vista vw_alumnos_membresias.

Escribe aqui tu consulta SQL:

Ejercicio 48 - CASE

Clasifica los pagos como Alto si son mayores o iguales a 800, Medio si son mayores o iguales a 500 y Bajo en caso contrario.

Escribe aqui tu consulta SQL:

Ejercicio 49 - CTE WITH

Usa una CTE para calcular total pagado por alumno y mostrar solo quienes hayan pagado mas de 700. Escribe aqui tu consulta SQL:

Ejercicio 50 - Transaccion basica

Escribe una transaccion que registre un pago nuevo para el alumno 1 y confirme los cambios. Escribe aqui tu consulta SQL:

Observacion: prueba la consulta y revisa que el resultado coincida con lo solicitado.

Rubrica global sugerida

Criterio	Excelente	Suficiente	Insuficiente	Ponderacion
Consultas basicas	Usa SELECT, WHERE, ORDER BY y filtros correctamente	Presenta errores menores	No obtiene el resultado solicitado	20%
Agregaciones	Aplica COUNT, SUM, AVG, GROUP BY y HAVING correctamente	Usa agregaciones con errores menores	Confunde agrupacion y filtrado	20%
JOIN	Relaciona tablas correctamente con PK/FK	Relaciona parcialmente	Produce combinaciones incorrectas	25%
Subconsultas y CTE	Usa subconsultas, EXISTS o WITH adecuadamente	Solucion parcial	No aplica el concepto	15%
Modificaciones	Ejecuta INSERT, UPDATE, DELETE o transacciones con seguridad	Requiere ajustes	Modifica datos incorrectamente	10%
Presentacion	Entrega ordenada y documentada	Entrega incompleta	Sin evidencias claras	10%

